



Data Visualization

LABORATORIO

Capitolo 7 – Grafi



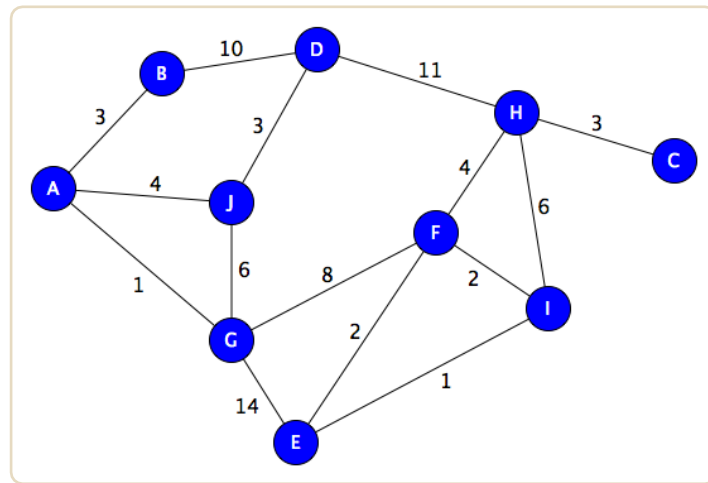
Argomenti del giorno

- **Grafi**
 - Struttura
 - Simulazioni force-directed

Perché usare i grafi

I grafi sono uno strumento estremamente potente per rappresentare **relazioni** tra oggetti e sono particolarmente utili in situazioni dove altri tipi di grafici (bar chart, line chart, scatter plot, etc.) non sono adatti.

Un grafo è una struttura composta da un insieme di **nodi** e un insieme di **archi** che collegano questi nodi. L'informazione principale non è il valore numerico contenuto nei nodi, ma la **relazione** tra essi.

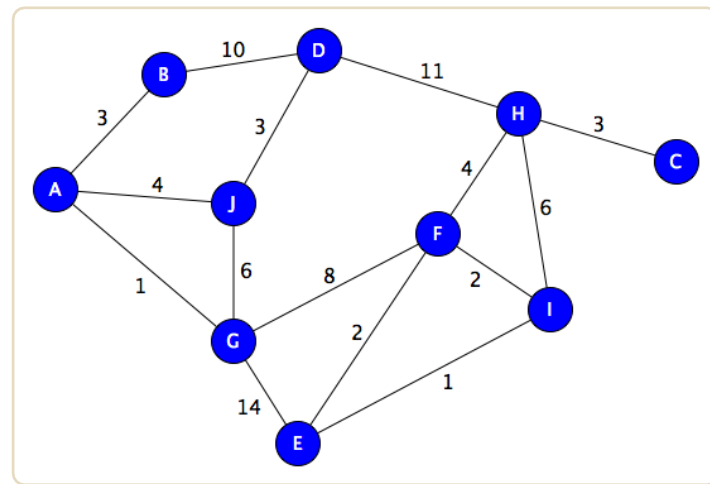


Quando usare i grafi

I grafi permettono di rappresentare **relazioni complesse** come connessioni multiple e non lineari tra entità, cosa difficile da fare con altri grafici.

Sono ideali per rappresentare reti di ogni tipo: sociali, informatiche, logistiche, etc.

Permettono infine analisi strutturali delle reti per trovare informazioni importanti.



Struttura dei dati

Diversamente dalle lezioni precedenti, in questo caso i dati per creare la visualizzazione sono in un file **.json** e non più .csv.

La struttura *{ chiave : valore }* dei file **JSON** li rende facilmente leggibili sia da umani che da macchine.

Nel nostro caso abbiamo una **lista di nodi** e **una di archi**. I nodi sono formati da un **id** e un **nome**, mentre gli archi da **sorgente** e **destinazione**, quindi gli id dei nodi di **partenza** e **arrivo** dell'arco.

```
async function getDataJSON(path){ Sho
  const data = await d3.json(path);
  console.log(data);
  return data;
}
```

```
"nodes": [
  {
    "id": 1,
    "name": "A"
  },
  {
    "id": 2,
    "name": "B"
  },
  {
    "id": 3,
    "name": "C"
  },
  {
    "id": 4,
    "name": "D"
  },
  {
    "id": 5,
    "name": "E"
  },
  {
    "id": 6,
    "name": "F"
  },
  {
```

```
"links": [
  {
    "source": 1,
    "target": 2
  },
  {
    "source": 1,
    "target": 5
  },
  {
    "source": 1,
    "target": 6
  },
  {
    "source": 2,
    "target": 3
  },
  {
    "source": 2,
    "target": 7
  },
  {
    "source": 3,
    "target": 4
  },
  {
```

Creazione nodi

Una volta caricati i dati e creato il solito svg, creiamo i nodi del nostro grafo.

Innanzitutto creiamo un **gruppo** per ogni **nodo** e successivamente aggiungiamo a ognuno di essi un **cerchio**.

```
// in posById si associa a ogni nodo la sua posizione dell'array nodesPosition
const posById = new Map(
  data.nodes.map((node, i) => [node.id, nodesPositions[i]])
);

const gContainer = svg
  .append('g')
  .attr('transform', `translate(${margin.left}, ${margin.top})`)
```

```
//un gruppo per ogni nodo
const nodesG = gContainer
  .selectAll(null)
  .data(data.nodes)
  .join("g")
  .classed('node', true)

// Per adesso, ogni nodo ha una posizione statica
// Spostiamo il gruppo (1 gruppo = 1 nodo)
.attr('transform', (d) =>
  `translate(${posById.get(d.id)[0]}, ${posById.get(d.id)[1]})`)

// Si crea un cerchio per ogni gruppo/nodo
const nodes = nodesG
  .append("circle")
  .attr("r", 20)
  .style("fill", "lightskyblue")
```

Creazione archi

Successivamente, ripetiamo lo stesso procedimento per gli **archi**.

Creiamo quindi un **gruppo** per ognuno di essi e poi aggiungiamo una **linea** che lo rappresenti visivamente.

```
// Un gruppo per arco
const linksG = gContainer
  .selectAll(null)
  .data(data.links)
  .join("g")
  .classed('link', true)

const arcs = linksG
  .selectAll("line")
  .data(d => [d]) // 1 linea per ogni gruppo-link
  .join("line")
  .style("stroke", "lightgray")
  .style("stroke-width", 2)
  .attr("x1", d => posById.get((d.source))[0])
  .attr("y1", d => posById.get((d.source))[1])
  .attr("x2", d => posById.get((d.target))[0])
  .attr("y2", d => posById.get((d.target))[1])
```

Label per i nodi e archi

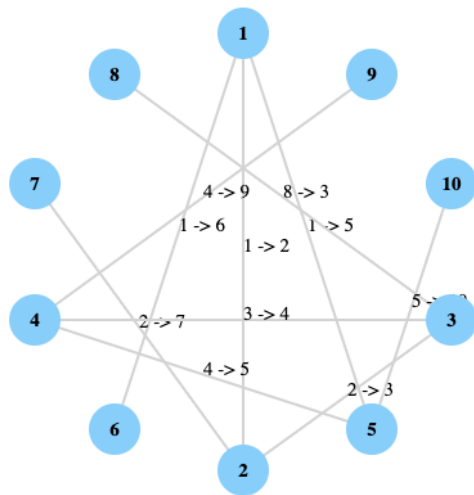
Le label dei nodi ci indicano gli id dei nodi.

Invece le label degli archi ci indicano il nodo sorgente e il nodo destinazione

```
const node_labels = nodesG
  .append("text")
  .text(d => d.id)
  .style("fill", "black")
  .style("font-size", 15)
  .style("font-weight", "bold")
  .attr("text-anchor", "middle")
  .attr("alignment-baseline", "central")
  .style("user-select", "none")

const arc_labels = linksG
  .append("text")
  .text(d => `${d.source} -> ${d.target}`)
  .style("font-size", 15)
  .attr("text-anchor", "start")
  .attr("alignment-baseline", "baseline")
  .style("user-select", "none")
  .attr("x", d => (posById.get(d.source)[0] + posById.get(d.target)[0]) / 2)
  .attr("y", d => (posById.get(d.source)[1] + posById.get(d.target)[1]) / 2)
```

Risultato



Modifiche sui nodi e archi

```
const arcs = linksG
  .selectAll("line")
  .data(d => [d]) // 1 linea per ogni gruppo-link
  .join("line")
  .style("stroke", "lightgray")
  .style("stroke-width", 2)
  .attr("x1", d => posById.get(d.source)[0])
  .attr("y1", d => posById.get(d.source)[1])
  .attr("x2", d => posById.get(d.target)[0])
  .attr("y2", d => posById.get(d.target)[1])
```



//la posizione degli archi sarà dinamica

```
const arcs = linksG
  .append("line")
  .style("stroke", "lightgray")
```

```
//un gruppo per ogni nodo
const nodesG = gContainer
  .selectAll(null)
  .data(data.nodes)
  .join("g")
  .classed('node', true)
// Per adesso, ogni nodo ha una posizione statica
// Spostiamo il gruppo (1 gruppo = 1 nodo)
.attr('transform', (d) =>
  `translate(${posById.get(d.id)[0]}, ${posById.get(d.id)[1]})`)
```



//un gruppo per ogni nodo ma non traslato (è dinamico adesso)

```
const nodesG = gContainer
  .selectAll(null)
  .data(data.nodes)
  .join("g")
  .classed('node', true)
```

Simulazione di forze

Tramite la funzione **d3.forceSimulation()** possiamo definire delle funzioni che simulano delle forze da applicare ai nostri elementi (simulazione **force-directed**).

È una fisica “**semplificata**” che D3 utilizza per posizionare gli elementi graficamente, calcolandone la posizione **dinamicamente** e **automaticamente** in modo tale che gli elementi non si sovrappongano.

Ogni nodo può essere soggetto a forze come **attrazione**, **repulsione**, **gravità** e **molle**.

```
function getSimulation(data, dimensions){ Show usages
  const [chartWidth, chartHeight] = dimensions;
  return d3.forceSimulation(data.nodes)
    .force("link", d3.forceLink()
      .id(d => d.id)
      .links(data.links)
      .distance(200)
    )
    .force("charge", d3.forceManyBody().strength(-400))
    .force("center", d3.forceCenter(chartWidth/2, chartHeight/2))
    .force("x", d3.forceX(chartWidth/2))
    .force("y", d3.forceY(chartHeight/2))
}
```

Simulazione di forze

La forza “**link**” simula una molla tra due nodi collegati da un arco (**link**). Tramite id si identificano i nodi e con links gli archi.

La forza “**charge**” aggiunge una carica repulsiva tra tutti i nodi.

La forza “**center**” invece sposta il centro gravitazionale, in questo caso al centro dello spazio a disposizione per il grafico.

Le forze “**x**” e “**y**” invece sono forze direzionali che spingono ogni nodo in una direzione fissa sugli assi x e y.

```
function getSimulation(data, dimensions){ Show usages
  const [chartWidth, chartHeight] = dimensions;
  return d3.forceSimulation(data.nodes)
    .force("link", d3.forceLink()
      .id(d => d.id)
      .links(data.links)
      .distance(200)
    )
    .force("charge", d3.forceManyBody().strength(-400))
    .force("center", d3.forceCenter(chartWidth/2, chartHeight/2))
    .force("x", d3.forceX(chartWidth/2))
    .force("y", d3.forceY(chartHeight/2))
}
```

Simulazione di forze

Dato che la simulazione è **iterativa**, bisogna definire i comportamenti degli elementi in base alle forze in gioco per ogni frame, definiti **ticks**.

Per farlo si **aggiornano** le coordinate, recuperandole da ogni collezione di dati.

In questo modo la funzione **.ticked()** aggiornerà i valori delle coordinate e disegnerà i nuovi layout.

```
rendergraph()
```

```
const simulation = getSimulation(data, [chartWidth, chartHeight])  
  .on("tick", () => {ticked(arcs, nodes, node_labels, arc_labels)});
```

```
function ticked(arcs, nodes, node_labels, arc_labels) { Show usages  
  
  // Aggiorna la posizione di ogni nodo.  
  nodes  
    .attr("cx", d => d.x)  
    .attr("cy", d => d.y)  
  
  // Aggiorna la posizione di ogni testo del nodo.  
  node_labels  
    .attr("x", d => d.x)  
    .attr("y", d => d.y)  
  
  //Aggiorna la posizione di ogni link.  
  arcs  
    .attr("x1", d => d.source.x)  
    .attr("y1", d => d.source.y)  
    .attr("x2", d => d.target.x )  
    .attr("y2", d => d.target.y);  
  
  // Aggiorna la posizione di ogni testo del link.  
  arc_labels  
    .attr("x", d => (d.source.x + d.target.x) / 2)  
    .attr("y", d => (d.source.y + d.target.y) / 2)  
}
```

Attivazione Drag & Drop

D3 permette di eseguire comportamenti particolari sugli elementi dei grafici. Tramite la funzione **d3.drag()** possiamo definire comportamenti personalizzati agli eventi del **Drag & Drop** (start, drag, end).

```
nodesG.call(d3.drag()  
  .on("start", dragstarted)  
  .on("drag", dragged)  
  .on("end", dragended));
```

Attivazione Drag & Drop

D3, riguardo alle simulazioni di forze, usa una terminologia relativa alla **fisica**.

drag started: vogliamo applicare un trascinamento, iniziamo “**riscaldando**” il nostro grafico tramite **alphaTarget()**. Aggiorniamo quindi le posizioni fisse del nodo con quelle del puntatore.

dragged: durante il trascinamento non abbiamo bisogno di aggiungere **energia** al sistema, pertanto basta solo mantenere **aggiornate** coordinate dei nodi

drag end: . Mentre alla fine del trascinamento diciamo al sistema che deve “**raffreddarsi**”, quindi l’**energia finale** deve tornare a **0**.

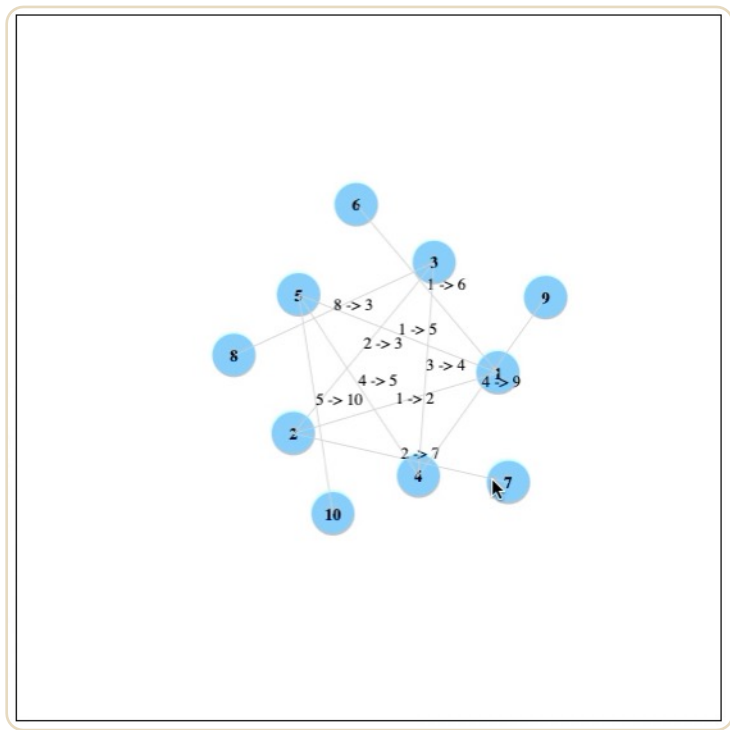
Annulliamo le coordinate fisse in modo tale che il nodo posso tornare a essere libero dalla forza esercitata dal trascinamento e quindi riassumerà le sue coordinate normali.

```
const dragstarted = (event) => { Show usages
  simulation.alphaTarget(0.3).restart();
  event.subject.fx = event.x;
  event.subject.fy = event.y;
}

const dragged = (event) => { Show usages
  event.subject.fx = event.x;
  event.subject.fy = event.y;
}

const dragended = (event) => { Show usages
  simulation.alphaTarget(0).restart();
  event.subject.fx = null;
  event.subject.fy = null;
}
```

Risultato



Esercizio

Svolgere l'esercizio 2b_graph_auto contenuto nell'archivio della lezione 6.

Costruire un grafico che mostri le relazioni tra le entità buyer, seller e car definite nel dataset.

Rappresentare gli archi della vendita, acquisto e passaggio di proprietà con stili diversi in modo da distinguere visivamente chi ha fatto cosa.

Aggiungere infine la simulazione di forze e il drag & drop.

